

Сценарии работы с форматом JsonML Richtext

Сценарий 1: Пользователь пишет текст в web-приложении или вставляет html-контент из другого сайта в наше web-приложение.

В пользовательском интерфейсе для работы с форматированным текстом применяют контрол [Богатый текстовый редактор](#) (коротко "БТР"). Он по умолчанию поддерживает автоматическое распознавание форматирования текста и санитизацию [небезопасных тегов](#) в целях исключения html-инъекций.

При копировании текста с внешнего сайта веб-браузер по умолчанию копирует его стиль (курсив, насыщенность, подчеркнутый, перечеркнутый). Формат скопированного текста — HTML.

Вставленный в БТР скопированный текст тут же автоматически конвертируется в формат JsonML Richtext, и в этом формате БТР отдаёт текст для отправки на БЛ. Для отображения вставленного текста в БТР выполняется обратная конвертация из JsonML Richtext в HTML Richtext, и полученный результат отображается через встроенный в БТР html-плеер — контрол RichEditor_base.TinyMCE.

Также вы можете работать с данными формата JsonML Richtext без контроля БТР:

- используйте контрол [Controls/decorator:Markup](#), чтобы вставить данные JsonML Richtext в TMPL/WML. Перед вставкой контента, контрол выполняет санитизацию ("вырезаются" все теги и атрибуты, кроме [разрешённых](#)).

```
<Controls.decorator:Markup value="{ { json } }" />
```

- используйте API класса [Controls/decorator:MarkupConverter](#), чтобы в исходном коде JavaScript самостоятельно вызывать конвертацию форматов HTML → JsonML и обратно.

Сценарий 2: Пользователь пишет plain text в мобильном приложении, и этот текст нужно преобразовать в Json на устройстве

Чтобы преобразовать Plain text в Json, в контроллере мобильного приложения используйте конвертер представлений PlainTextBuilder.

Для преобразования вызовите метод ToJson, принадлежащий классу PlainTextBuilder.

Обычный текст (Plain text) преобразуется с выделением ссылок и сохранением переносов, пробелов и табуляций в Json.

- C++

```
1 #include <sbis-xml-json-converter/plain_text_builder.hpp>
2 String plain_text = LR"Link http://www.google.com/ text";
3 String result_json = PlainTextBuilder::ToJson( plain_text );
4
5 // result_json is LR"[{"p", { "version": "2" }, "Link ", [{"a", {
  "href": "http:\\\\www.google.com\\" }, "http:\\\\www.google.com\\"}],
  " text"]]"
```

- Python

```
1 import xml_json
2 ptb = xml_json.PlainTextBuilder()
3 text = 'Link http://www.google.com/ text'
4 json = ptb.ToJson( text )
5
```

```
6 # json is '[[{"p", { "version": "2" }, "Link ", [{"a", { "href":
  "http:\\\\www.google.com\\/", "target": "_blank" },
  "http:\\\\www.google.com\\/"}], " text"]]'
```

Сценарий 3: Сервис СБИС получает HTML-контент и сохраняет его в JsonML Richtext поле

Чтобы сохранить данные формата HTML в поле формата JsonML Richtext, нужно конвертировать данные из одного формата в другой. Для этого используйте методы `Convert()` и `ConvertBatch()` из класса `sbis::jsonxml::XmlJsonConverter`. Класс является частью конвертера представлений XML-JsonML, который следует подключить в прикладной проект согласно [руководству](#).

Если html-данные представлены одной строкой, используйте метод `Convert()`.

- Пример на C++

```
1 #include <sbis-xml-json-converter/xml_json.hpp>
2 XmlJsonConverter converter;
3 String richtext_source = LR"===(<span style="color:
  blue;">Текст</span>)===";
4 String result_json = converter.Convert( richtext_source );
5 // result_json is LR"===([["span", { "style": "color: blue;" },
  "Текст"]])==="
```

- Пример на Python

```
1 import xml_json
2 xjc = xml_json.XmlJsonConverter( xml_json.XJErrorLevel.lError )
3 xml = '<span style="color: blue;">Текст</span>'
4 json = xjc.Convert( xml )
5 # json is '[[{"span", { "style": "color: blue;" }, "Текст"}]]'
```

Если html-данные представлены массивом строк, используйте метод `ConvertBatch()`.

- Пример на C++:

```
1 #include <sbis-xml-json-converter/xml_json.hpp>
2 XmlJsonConverter converter;
3 ArrayString xml( 2 );
4 xml[ 0 ] = LR"===(<span style="color: blue;">Текст</span>)===";
5 xml[ 1 ] = LR"===(<div>test</div>)===";
6 ArrayString result = converter.ConvertBatch( xml );
7 //result[ 0 ] is LR"===([["span",{"version":"1.0", "style": "color:
  blue;" }, "Текст"]])===
8 //result[ 1 ] is LR"===([["div", "test"]])==="
```

- Пример на Python

```
1 import xml_json
2 xjc = xml_json.XmlJsonConverter( xml_json.XJErrorLevel.lError )
3 arr_xml = [ '<div>sometext</div>', '<span style="color:
  blue;">Текст</span>' ]
4 arr_json = xjc.ConvertBatch( arr_xml )
5 # arr_json is [ '[[{"div", "sometext"}]]', '[[{"span", { "style":
  "color: blue;" }, "Текст"}]]' ]
```

Сценарий 4: Конвертация данных из формата HTML Richtext в данные формата JsonML Richtext

"Ленивая" конвертация данных HTML Richtext → JsonML Richtext

Осуществляется над конкретными данными приложения, благодаря чему происходит постепенный переход к новому формату. В этом сценарии подразумевается, что в приложении поддерживается работа с данными обоих форматов.

Шаг 1: Создайте поле для хранения данных формата JsonML Richtext.

На время перехода к новому формату в БД должны существовать два поля: одно для данных формата HTML Richtext, а второе — для данных JsonML Richtext. В процессе конвертации поле JsonML Richtext будет заполняться новыми данными.

Шаг 2: Добавьте в бизнес-логику конвертер представлений XML-JsonML

С помощью такого конвертера можно преобразовывать данные из одного формата в другой. Конвертер следует подключить в прикладной проект согласно [руководству](#). API конвертера представлено в [разделе](#).

Если данные для конвертации отсутствуют, то поле JsonML Richtext следует заполнять пустой структурой — [].

```
1 if one_doc.Get('РазличныеДокументы.ИнформацияJSON', None) is None:
2     comment = one_doc.Get('РазличныеДокументы.Информация', None)
3     one_doc.Set('РазличныеДокументы.ИнформацияJSON',
        XmlToJson.Convert(comment) if comment else "[]")
```

В целях обратной совместимости на время перехода к новому формату, в БД сохраняйте данные как в формате JsonML Richtext, так и в HTML Richtext.

В ряде задач требуется обработка RecordSet'a, у которого есть:

- поле, в котором лежит HTML и которое надо конвертировать в JSON;
- поле, в котором должна лежать JSON модель с данными.

Рекомендуем [ознакомиться с реализацией](#), где осуществляется подобная конвертация форматов.

Шаг 3: Откажитесь от использования поля HTML Richtext

Происходит после конвертации в новый формат всех данных приложения.

Массовая конвертация HTML Richtext → JsonML Richtext



Потенциально опасная операция!

Важно понимать какой объёмом данных будет обработан. Массовая конвертация допустима над несколькими тысячами записей. Когда обрабатываются сотни тысяч записей, применяйте ["ленивую" конвертацию](#).

Выполняется сразу над всеми (целевыми) данными приложения. В приложении online.sbis.ru массовая конвертация осуществляется с помощью скрипта, а его выполнение на серверах заявляется по регламенту [Выполнить на рабочем](#).

Сценарий 5: Преобразовать данные формата JsonML Richtext версии X в формат версии Y

Преобразование возможно только к последней версии JsonML Richtext, а к предыдущим — не поддерживается. Версия, в которую конвертируются данные, определяется конвертером представлений XML-JsonML.

Преобразование к новой версии формата JsonML Richtext может происходить только на стороне бизнес-логики приложения. На клиентской стороне контрол "БТР" работает всегда с одной версией формата — той, которая пришла с бизнес-логики. Подробнее об этом можно прочитать в разделе ["Версии формата JsonML"](#).

"Ленивая" конвертация данных в новую версию формата

Осуществляется над конкретными данными приложения, благодаря чему происходит постепенный переход к новой версии формата JsonML Richtext.

Для конвертации используются методы TryConvert() и TryConvertBatch() из класса sbis::jsonxml::XmlJsonConverter. Класс является частью конвертера представлений XML-JsonML, который следует подключить в прикладной проект согласно [руководству](#).

Если данные представлены одной строки, используйте метод TryConvert().

- Пример на C++

```
1 #include <sbis-xml-json-converter/xml_json.hpp>
2 XmlJsonConverter converter;
3 String in_json = LR"===([["span", {"version":"1.0", "style": "color:
  blue;" }, "Текст"]])===";
4 String result = converter.TryConvert( in_json );
5 // result is LR"===([["span", {"version":"2.0", "style": "color:
  blue;" }, "Текст"]])==="
```

- Пример на Python

```
1 import xml_json
2 xjc = xml_json.XmlJsonConverter( xml_json.XJErrorLevel.lError )
3 json = '[[["span", {"version": "1.0", "style": "color: blue;" },
  "Текст"]]]'
4 res = xjc.TryConvert( json )
5 # res is '[[["span", {"version": "2.0", "style": "color: blue;" },
  "Текст"]]]'
```

Если данные представлены массивом строк, используйте метод TryConvertBatch().

Массовая конвертация данных в новую версию формата



Потенциально опасная операция!

Важно понимать какой объёмом данных будет обработан. Массовая конвертация допустима над несколькими тысячами записей. Когда обрабатываются сотни тысяч записей, применяйте ["ленивую" конвертацию](#).

Выполняется сразу над всеми (целевыми) данными приложения. В приложении [online.sbis.ru](#) массовая конвертация осуществляется с помощью скрипта, а его выполнение на серверах заявляется по регламенту [Выполнить на рабочем](#).

Сценарий 6: Преобразовать JsonML Richtext в plain text (без верстки)

Ещё одним из частых сценариев является получение текста из поля формата JsonML Richtext и последующая линейаризация (исключение тегов, описывающих форматирование). Сценарий встречается, например, для подготовки данных для рендера в реестре или уведомлений.

Для конвертации используйте методы ExtractText() и ExtractTextBatch() из класса sbis::jsonxml::JsonXmlConverter. Класс является частью конвертера представлений XML-JsonML, который следует подключить в прикладной проект согласно [руководству](#).

Если данные представлены одной строки формата JsonML Richtext, используйте метод ExtractText().

- Пример на C++

```
1 #include <sbis-xml-json-converter/json_xml.hpp>
2 JsonXmlConverter converter;
3 String json = LR"===([["span", "sometext"]])===";
4 String result = converter.ExtractText( json );
5 //result is "sometext"
```

- Пример на Python

```
1 import xml_json
2 jxc = xml_json.JsonXmlConverter()
3 json = '[[["span", { "style": "color: blue;" }, "Текст"]]]'
4 text = jxc.ExtractText( json )
```

```
5 | # text is 'Текст'
```

Если данные представлены массивом строк формата JsonML Richtext, используйте метод `ExtractTextBatch()`.

- Пример на C++

```
1 | #include <sbis-xml-json-converter/json_xml.hpp>
2 | JsonXmlConverter converter;
3 | StringArray json( 2 );
4 | json[ 0 ] = LR"==(["span", "sometext"])==";
5 | json[ 1 ] = LR"==(["span", { "style": "color: blue;" },
   | "Текст"])==";
6 | StringArray result = converter.ExtractTextBatch(json);
7 | // result[ 0 ] is "sometext"
8 | // result[ 1 ] is "Текст"
```

- Пример на Python

```
1 | import xml_json
2 | jxc = xml_json.JsonXmlConverter()
3 | json = [ '["span", { "style": "color: blue;" }, "Текст"]',
   | '["span", "sometext"]' ]
4 | xml = jxc.ExtractTextBatch( json )
5 | # xml is [ 'Текст', 'sometext' ]
```

Сценарий 7: Индексация JsonML Richtext для полнотекстового поиска PostgreSQL.

Текст, конвертированный в формат JsonML Richtext, может участвовать в [полнотекстовом поиске PostgreSQL](#). Чтобы поддержать работу поиска, текст должен быть [проиндексирован](#). Для решения этой задачи Wasaby Framework предоставляет прикладным разработчикам системную функцию `sbis_rt_filter`. С помощью этой функции можно отфильтровать "богатый" текст, исключив теги разметки, а результат — использовать для индексации в поиске.

Имя параметра	Тип	Описание
rt_doc	text	Текст в формате JsonML RichText. ["test doc", ["p", "par1"]]
fts_cfg_name	regconfig	fts-конфигурация для преобразования фильтрованного текста в tsvector. Параметр опциональный. По умолчанию simple. Подробнее про конфигурацию вы можете прочитать по ссылке .

Подключение системной функции в прикладной проект не требуется. Она разворачивается автоматически [конвертером БД](#) в схему public.

Имена системных функций формируются следующим образом:

`sbis_<имя функции>_v_<номер версии функции>`

При изменении реализации функции добавляется функция с таким же именем и повышенным номером версии, например `sbis_rt_filter_v2`. При этом функция предыдущей версии `sbis_rt_filter_v1` также будет существовать, но в следующих релизах будет удалена ответственным программистом — [Баранов Е.Ю.](#). Время удаления согласовывается со всеми разработчиками в опубликованной на [online.sbis.ru](#) новости.

[СТАН](#) получает список актуальных системных функций и их версий через механизм свойств алгоритма конвертации. Каждое условие индекса в проекте будет проверено на наличие имени системной функции и ее версии по шаблону формирования имен системных функций (см. выше). В случае обнаружения использования устаревшей системной функции будет сгенерирована [ошибка СТАН](#).

При получении такой ошибки необходимо перевести условие индекса на новую версию системной функции, отредактировав выражение проблемного индекса. Далее необходимо оценить объём индексируемых данных, так как пересоздание индекса — процедура не быстрая.

Если количество записей в таблице менее 1000, или вы уверены в скорости хранилища, то на этом процедура изменений завершена, так как на [первой фазе конвертации БД](#) новый индекс будет создан автоматически в параллельном режиме.

Если записей много или нет возможности оценить объёмы, то необходимо создать скрипт с помощью [утилиты](#) и отправить его на выполнение через [ВНР](#). Данная утилита была разработана специально для случаев, когда есть "долгие" индексы, которые необходимо создать до начала обновления БД сервиса, чтобы не затягивать предварительную фазу конвертации БД.

Примеры использования

Создание индекса через SQL-запрос:

```
create index "iTable-Index" on "Table" using gin(sbis_rt_filter_v1("RtFld"))
where "RtFld" is not null;
```

или аналогичная запись

```
create index "iTable-Index1" on "Table" using gin(sbis_rt_filter_v1("RtFld",
'ruussian'::regconfig)) where "RtFld" is not null;
```